

Практическое занятие 3.1

Основы TensorFlow и работа с тензорами

Цели работы

1. Освоить базовые понятия библиотеки **TensorFlow**: тензор, форма, тип данных, ранг.
2. Научиться создавать и изменять тензоры, выполнять математические операции.
3. Изучить принцип автоматического дифференцирования (`tf.GradientTape`).
4. Освоить реализацию **градиентного спуска** и исследовать влияние шага обучения на сходимость.

Краткие теоретические сведения

TensorFlow — это платформа машинного обучения с открытым исходным кодом, разработанная в Google.

Она позволяет выполнять численные операции с многомерными массивами — **тензорами**, автоматически вычислять производные функций и использовать вычисления на GPU и TPU.

Тензор — многомерная структура данных, аналог массивов NumPy, но с возможностью выполнения вычислений на различных устройствах.

Каждый тензор имеет:

- **ранг** — число осей (скаляр = 0, вектор = 1, матрица = 2, трёхмерный тензор = 3 и т.д.);
- **форму (shape)** — размеры по каждой оси (например, (3, 4));
- **тип данных (dtype)** — например, `tf.float32`, `tf.int32`.

TensorFlow обеспечивает как **поэлементные** операции (сложение, умножение, возведение в степень),

так и **матричные** (например, `tf.matmul`), а также автоматическое вычисление градиентов функций.

1. Подключение библиотек и проверка версий

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow version:", tf.__version__)
print("NumPy version:", np.__version__)
```

Комментарий:

TensorFlow использует динамическое вычисление (*eager execution*), что означает — код выполняется сразу, а не через построение графа (как в версии 1.x).

2. Создание и свойства тензоров

Пример 1. Создание тензоров разных рангов

```
a = tf.constant(5) # скаляр
b = tf.constant([1, 2, 3]) # вектор
c = tf.constant([[1.0, 2.0], [3.0, 4.0]]) # матрица

for name, t in zip(["a", "b", "c"], [a, b, c]):
    print(f"{name}: rank={tf.rank(t).numpy()}, shape={t.shape},
dtype={t.dtype}")
```

Результат:

- a — ранг 0 (скаляр);
- b — ранг 1 (вектор длины 3);
- c — ранг 2 (матрица 2×2).

Контрольные вопросы

1. Что такое ранг тензора?
2. Как изменить тип данных тензора (`dtype`)?
3. Можно ли создать тензор из списка списков разной длины?

3. Создание тензоров разными способами

```
# Константы
a = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)

# Нули и единицы
z = tf.zeros((2, 3))
o = tf.ones((2, 3))

# Случайные значения
r = tf.random.normal((2, 3), mean=0.0, stddev=1.0)

print("a:\n", a.numpy())
print("zeros:\n", z.numpy())
print("ones:\n", o.numpy())
print("random normal:\n", r.numpy())
```

Пояснение:

- `tf.zeros` и `tf.ones` — генерируют тензоры с заданной формой.
- `tf.random.normal` создаёт тензор со случайными значениями из нормального распределения.

4. Операции с тензорами

```
x = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
```

```

y = tf.constant([[5, 6],[7, 8]], dtype=tf.float32)

add = tf.add(x, y)
mul = tf.multiply(x, y)
dot = tf.matmul(x, y)

print("Сумма:\n", add.numpy())
print("Позлементное умножение:\n", mul.numpy())
print("Матричное произведение:\n", dot.numpy())

```

Различие:

- `tf.multiply` — поэлементное умножение (аналог `*` в NumPy);
- `tf.matmul` — матричное умножение (линейная алгебра).

Мини-исследование 1. Скорость умножения больших матриц

Измерим время выполнения операции `tf.matmul` при разных размерах матриц.

```

import time

for n in [100, 500, 1000, 2000]:
    A = tf.random.normal((n,n))
    B = tf.random.normal((n,n))
    start = time.time()
    _ = tf.matmul(A,B)
    print(f"Размер {n}x{n}, время: {time.time()-start:.4f} сек.")

```

Задание:

1. Заполните таблицу размер-время.
2. Постройте график в Excel или Matplotlib.
3. Сделайте вывод о зависимости времени от размера матрицы.

5. Переменные и автоматическое дифференцирование

TensorFlow позволяет автоматически вычислять производные (градиенты). Для этого используется контекст `tf.GradientTape`.

Пример 2. Вычисление градиента функции ($y = x^2 + 2x + 1$)

```

x = tf.Variable(3.0)

with tf.GradientTape() as tape:
    y = x**2 + 2*x + 1

dy_dx = tape.gradient(y, x)
print("x =", x.numpy(), " dy/dx =", dy_dx.numpy())

```

Ожидаем: $dy/dx = 2x + 2 = 8.0$

`GradientTape` «запоминает» операции внутри блока и автоматически применяет правило цепочки.

Контрольные вопросы

- Что делает `GradientTape()`?
- Как TensorFlow определяет, по каким переменным считать производные?
- Почему не все операции могут быть дифференцируемыми?

6. Градиентный спуск для функции $L(w) = (w - 3)^2$

```
w = tf.Variable(0.0)
alpha = 0.1 # шаг обучения
history = []

for step in range(30):
    with tf.GradientTape() as tape:
        loss = tf.square(w - 3.0)
        grad = tape.gradient(loss, w)
        w.assign_sub(alpha * grad)
        history.append(float(loss))
    if step % 5 == 0:
        print(f"step={step:02d}, w={w.numpy():.4f}, loss={float(loss):.6f}")

print("Final w:", w.numpy())
```

Результат: постепенное приближение к минимуму при $w \rightarrow 3$.

Визуализация сходимости

```
plt.plot(history)
plt.title("Снижение функции потерь при градиентном спуске")
plt.xlabel("Итерация"); plt.ylabel("Loss"); plt.show()
```

7. Исследование влияния шага обучения (α)

Проведите эксперимент для $\alpha = 0.01, 0.1, 0.5, 1.0$ и постройте графики изменения loss .

```
def run_gd(alpha=0.1, steps=40):
    w = tf.Variable(0.0)
    hist = []
    for _ in range(steps):
        with tf.GradientTape() as tape:
            loss = tf.square(w - 3.0)
            grad = tape.gradient(loss, w)
            w.assign_sub(alpha * grad)
            hist.append(float(loss))
    return w.numpy(), hist

plt.figure(figsize=(6,4))
for a in [0.01, 0.1, 0.5, 1.0]:
    w_final, h = run_gd(alpha=a)
    plt.plot(h, label=f" $\alpha={a}$ ", w_final={w_final:.3f})
plt.legend(); plt.xlabel("step"); plt.ylabel("loss"); plt.title("Влияние шага обучения"); plt.show()
```

Наблюдения:

- При слишком маленьком $\alpha \rightarrow$ сходимость медленная.
- При слишком большом $\alpha \rightarrow$ возможна расходимость («скачки» вокруг минимума).
- Оптимальный α зависит от формы функции и масштаба данных.

8. Сравнение с ручным градиентом

```
w = 0.0
alpha = 0.1
for step in range(10):
    grad = 2*(w - 3.0)
    w -= alpha * grad
    print(f"step={step}, w={w:.4f}, loss={(w-3)**2:.6f}")
```

Вывод: Результаты совпадают с автоматическим градиентом.

Auto-diff освобождает от необходимости ручных выводов для сложных сетей.

9. Исследовательская задача для отчёта

Задача: исследовать зависимость скорости и качества сходимости от шага обучения и числа итераций.

1. Напишите функцию `train(alpha, steps)`, возвращающую историю `loss`.
2. Проведите эксперименты с разными α и `steps`.
3. Постройте графики на одной оси.
4. Сделайте 3–5 тезисов по результатам (в отчёте).

Контрольные вопросы

1. Что такое тензор и его основные свойства?
2. В чём разница между `tf.constant` и `tf.Variable`?
3. Что делает `GradientTape` и как он вычисляет производные?
4. Почему важен шаг обучения в градиентном спуске?
5. Как проверить, что модель достигла минимума функции потерь?

Структура отчёта по практической работе

1. Титульный лист (название работы, Ф.И.О., дата).
2. Цель работы.
3. Краткие теоретические сведения.
4. Ход выполнения (фрагменты кода, результаты экспериментов, графики).
5. Ответы на контрольные вопросы.
6. Выводы (3–5 пунктов).

Выводы

- TensorFlow — гибкая платформа для работы с тензорами и реализации алгоритмов обучения.
- Механизм `GradientTape` позволяет автоматически находить градиенты и упрощает написание оптимизационных циклов.
- Градиентный спуск является базой для обучения нейронных сетей.

- Корректный выбор шага обучения определяет эффективность и устойчивость обучения.

Практика 3.1 — Основы TensorFlow и работа с тензорами

Цели:

1. Освоить понятия *тензор*, *ранг*, *форма (shape)*, *тип данных (dtype)*.
2. Научиться создавать константы и переменные в TensorFlow.
3. Понять механизм автоматического дифференцирования (`tf.GradientTape`).
4. Экспериментально исследовать градиентный спуск и влияние шага обучения.

Ожидаемые результаты: студент умеет создавать тензоры, вычислять градиенты и реализовывать простой градиентный спуск, понимать влияние гиперпараметров на сходимость.

```
In [ ]: # Импорт и проверка версий
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

print("TensorFlow version:", tf.__version__)
print("NumPy version:", np.__version__)
```

1. Что такое тензор: ранг, форма, тип данных

Тензор — это обобщение скаляра (ранг 0), вектора (ранг 1), матрицы (ранг 2) и тензоров более высоких рангов.

- **Ранг (rank)** — число осей.
- **Форма (shape)** — размеры по каждой оси.
- **Тип данных (dtype)** — например, `tf.float32`, `tf.int32`.

Задание: Запустите ячейку ниже и объясните, почему ранги равны 0, 1 и 2 соответственно.

```
In [ ]: # Пример: скаляр, вектор, матрица
a = tf.constant(5) # ранг 0
b = tf.constant([1, 2, 3]) # ранг 1
c = tf.constant([[1.0, 2.0], [3.0, 4.0]]) # ранг 2

for name, t in zip(["a", "b", "c"], [a, b, c]):
    print(f"{name}: rank={tf.rank(t).numpy()}, shape={t.shape}, dtype={t.dtype}")
```

Контрольные вопросы

1. Чем отличается скаляр от вектора и матрицы с точки зрения ранга?

2. Как изменить тип данных тензора? Приведите пример преобразования `int32` -> `float32` .

2. Создание тензоров и базовые операции

Ниже показано создание констант, тензоров нулей/единиц, случайных тензоров, а также поэлементные и матричные операции.

```
In [ ]: # Создание тензоров
a = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
z = tf.zeros((2,3))
o = tf.ones((2,3))
r = tf.random.normal((2,3), mean=0.0, stddev=1.0)

print("a:\n", a.numpy())
print("zeros:\n", z.numpy())
print("ones:\n", o.numpy())
print("random normal:\n", r.numpy())

# Поэлементные операции
x = tf.constant([[1, 2], [3, 4]], dtype=tf.float32)
y = tf.constant([[5, 6], [7, 8]], dtype=tf.float32)

add = tf.add(x, y)
mul = tf.multiply(x, y)
dot = tf.matmul(x, y)

print("\nСумма:\n", add.numpy())
print("Поэлементное умножение:\n", mul.numpy())
print("Матричное произведение:\n", dot.numpy())
```

Мини-исследование 1: масштаб матриц и время вычислений

Изучите влияние размера матриц на время умножения. Сделайте вывод о сложности операции и производительности.

```
In [ ]: import time

sizes = [100, 500, 1000]
times = []
for n in sizes:
    A = tf.random.normal((n,n))
    B = tf.random.normal((n,n))
    t0 = time.time()
    _ = tf.matmul(A,B)
    dt = time.time() - t0
    times.append(dt)
    print(f"Размер {n}x{n}, время: {dt:.4f} сек.")

# Выводы (заполните после выполнения):
```



```
# 1) ...
# 2) ...
# 3) ...
```

3. Переменные (`tf.Variable`) и автоматическое дифференцирование

Внутри контекста `tf.GradientTape()` TensorFlow записывает операции и может вычислить производные (градиенты) по указанным переменным.

```
In [ ]: # Пример:  $y = x^2 + 2x + 1$ ,  $dy/dx = 2x + 2$ 
x = tf.Variable(3.0)

with tf.GradientTape() as tape:
    y = x**2 + 2*x + 1

dy_dx = tape.gradient(y, x)
print("x =", x.numpy(), " dy/dx =", dy_dx.numpy()) # ожидаем 8.0
```

Контрольные вопросы

- Что делает контекст `GradientTape` ?
- Как TensorFlow «знает», какие операции дифференцировать?

4. Градиентный спуск для $L(w) = (w - 3)^2$

Сходимость к минимуму определяется шагом обучения `alpha`. Ниже — реализация с автоматическим градиентом.

```
In [ ]: w = tf.Variable(0.0)
alpha = 0.1
history = []

for step in range(30):
    with tf.GradientTape() as tape:
        loss = tf.square(w - 3.0)
        grad = tape.gradient(loss, w)
        w.assign_sub(alpha * grad) #  $w \leftarrow w - \alpha * grad$ 
        history.append(float(loss))
    if step % 5 == 0:
        print(f"step={step:02d}, w={w.numpy():.4f}, loss={float(loss):.6f}")

print("Final w:", w.numpy())
```

```
In [ ]: # Визуализация сходимости
plt.figure(figsize=(5,3))
plt.plot(history)
plt.title("Изменение функции потерь при градиентном спуске")
plt.xlabel("Итерация")
```

```
plt.ylabel("Loss")
plt.show()
```

Мини-исследование 2: влияние шага обучения

1. Повторите эксперимент для `alpha = 0.01, 0.1, 0.5, 1.0`.
2. Постройте графики loss на одной оси.
3. Объясните, когда и почему алгоритм расходится.

```
In [ ]: def run_gd(alpha=0.1, steps=30):
    w = tf.Variable(0.0)
    hist = []
    for _ in range(steps):
        with tf.GradientTape() as tape:
            loss = tf.square(w - 3.0)
            grad = tape.gradient(loss, w)
            w.assign_sub(alpha * grad)
            hist.append(float(loss))
    return w.numpy(), hist

plt.figure(figsize=(6,4))
for a in [0.01, 0.1, 0.5, 1.0]:
    w_final, h = run_gd(alpha=a, steps=40)
    plt.plot(h, label=f"alpha={a}, w_final={w_final:.3f}")
plt.legend(); plt.xlabel("step"); plt.ylabel("loss"); plt.title("Влияние alpha"); p

# Выводы (заполните):
# - При слишком большом alpha наблюдается ...
# - При слишком маленьком alpha ...
# - Оптимальный диапазон alpha для данной задачи ...
```

5. Ручной градиентный спуск (без GradientTape)

Покажем эквивалентность с автоматическим дифференцированием на простой функции.

```
In [ ]: w = 0.0
alpha = 0.1
for step in range(10):
    grad = 2*(w - 3.0)      # аналитический градиент (w-3)^2
    w = w - alpha * grad
    print(f"step={step:02d}, w={w:.4f}, loss={(w-3.0)**2:.6f}")
print("Final w:", w)
```

6. Исследовательская задача (для отчёта)

Задача: изучить, как выбор `alpha` и число итераций влияют на качество и скорость сходимости.

- Реализуйте функцию `train(alpha, steps)` (шаблон выше).
- Проведите серию экспериментов и постройте сводный график.
- Сформулируйте 3–5 тезисов по результатам.

Контрольные вопросы

1. Почему при слишком большом `alpha` возможна расходимость?
2. Что происходит с сходимостью при `alpha -> 0` ?
3. Чем авто-диф удобнее ручного вычисления градиента?